

Discrete Optimal Trajectory Planning on Curved Track Geometries via Bellman Value Iteration

Autonomous Racing Systems

Aditya Vinay Pathange

Bachelor's in Interdisciplinary Engineering

August 10, 2026

Abstract

Determining optimal racing trajectories within spatial boundaries is a fundamental problem in autonomous navigation and trajectory optimization. While classical control approaches rely on physics-informed kinematic models, model-based dynamic programming provides a robust framework for global trajectory extraction across discrete grid representations. In this work, we propose a discrete Reinforcement Learning (RL) approach leveraging the Markov Decision Process (MDP) formulation and Bellman Value Iteration to compute optimal velocity profiles and cornering trajectories on a synthetic S-shaped track geometry. We define a discrete state-action space incorporating velocity bounds, directional steering offsets, boundary collision penalties, and speed-dependent turning costs. Our algorithm converges within 20 offline iterations, successfully generating optimal braking points, acceleration zones, and trajectory lines rendered via real-time visualization. We analyze the policy's emergent behaviors, convergence criteria, and structural limitations resulting from non-physics-informed discrete state approximations.

1 Introduction

Autonomous vehicle control and motorsport engineering both require calculating optimal path trajectories that minimize time-to-finish while respecting boundary limits and mechanical friction constraints. Modern trajectory optimization often employs non-linear model predictive control (NMPC) or physics-informed differential equations to model tire dynamics, lateral slip, and aerodynamic downforce. However, these continuous frameworks incur substantial computational complexity and require exact parameter tuning.

Discrete dynamic programming, rooted in the foundational work of Richard Bellman, offers an exact, global optimization mechanism for Markov Decision Processes (MDPs) over discretized state spaces. By dynamic discretization of position and velocity states, the continuous racing line problem transforms into an exact tabular optimal control task.

In this paper, we present a simplified grid-based reinforcement learning environment designed to solve the trajectory and speed-profile optimization problem on a continuous-like S-curve geometry. Using Bellman Value Iteration, the agent evaluates combined actions for speed variation and lateral control under customized penalization schemes.

1.1 Key Contributions

- **Grid Environment Formalization:** A parameterized track-generation process using spatial sine-wave functions mapped onto a discrete grid matrix.
- **Joint State Space Dynamics:** Formulation of a combined state tuple $s = (x, y, v)$ governing both spatial navigation and variable dynamic velocity bounds.

- **Action-Dependent Friction Approximations:** Implementation of velocity-penalized steering costs within the reward dynamics to emulate loss of traction at higher speeds without explicit non-linear dynamic physics equations.
- **Real-Time Visual Policy Rendering:** An interactive Pygame visualization displaying spatial waypoints color-coded by dynamic policy decisions (acceleration, braking, and maintenance).

—

2 Markov Decision Process (MDP) Formulation

We model the track traversal process as a discrete-time stationary Markov Decision Process defined by the 5-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

2.1 State Space ($\mathcal{S}$)

The spatial environment is bounded by a grid of dimensions $W \times H$, where $W = \lfloor \text{Width}/g \rfloor$ and $H = \lfloor \text{Height}/g \rfloor$ with grid cell resolution $g = 15$ px. Each state $s \in \mathcal{S}$ is represented by a tuple:

$$s = (x, y, v) \in \{0, \dots, W-1\} \times \{0, \dots, H-1\} \times \{v_{\min}, \dots, v_{\max}\} \quad (1)$$

where $v_{\min} = 1$ and $v_{\max} = 6$. The occupancy matrix $\mathbf{T}_{x,y} \in \{0, 1, 2\}$ denotes whether a coordinate represents a Wall (0), Track (1), or Finish Line (2).

2.2 Action Space ($\mathcal{A}$)

The agent selects joint actions $a = (a_v, a_y) \in \mathcal{A}$ controlling longitudinal acceleration and lateral position shift simultaneously:

$$a_v \in \{-1, 0, 1\} \quad (\text{Brake, Maintain, Accelerate}) \quad (2)$$

$$a_y \in \{-1, 0, 1\} \quad (\text{Steer Up, Straight, Steer Down}) \quad (3)$$

2.3 Transition Dynamics ($\mathcal{P}$)

The system state updates deterministically according to the chosen joint action $a = (a_v, a_y)$ at step t :

$$v_{t+1} = \text{clip}(v_t + a_v, v_{\min}, v_{\max}) \quad (4)$$

$$x_{t+1} = x_t + v_{t+1} \quad (5)$$

$$y_{t+1} = y_t + a_y \quad (6)$$

If $x_{t+1} \geq W$ or $y_{t+1} < 0$ or $y_{t+1} \geq H$, the transition terminates out-of-bounds.

2.4 Reward Function ($\mathcal{R}$)

To incentivize fast completion while preventing boundary collisions and high-speed lateral instability, the scalar reward $R(s, a, s')$ is defined conditionally:

$$R(s, a, s') = \begin{cases} +100.0 & \text{if } \mathbf{T}_{x',y'} = 2 \quad (\text{Goal reach}) \\ -50.0 & \text{if } \mathbf{T}_{x',y'} = 0 \quad (\text{Collision/Off-track}) \\ -1.0 - c_{\text{turn}} & \text{otherwise} \end{cases} \quad (7)$$

where the dynamic turn cost c_{turn} acts as a heuristic surrogate for centrifugal force and traction loss:

$$c_{\text{turn}} = \begin{cases} 0.5 \cdot |a_y| & \text{if } v_{t+1} = v_{\text{max}} \\ 0.0 & \text{otherwise} \end{cases} \quad (8)$$

3 Methodology: Bellman Value Iteration

To determine the optimal dynamic behavior across all track locations and velocities, we compute the optimal state-value function $V^*(s)$ using offline Value Iteration derived from the Bellman Optimality Equation:

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \mathcal{P}(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \quad (9)$$

Given deterministic transitions, the update rule simplifies to:

$$V_{k+1}(x, y, v) = \max_{(a_v, a_y)} \{R(s, a, s') + \gamma V_k(x', y', v')\} \quad (10)$$

where $\gamma = 0.95$ represents the discount factor prioritizing shorter time-to-target paths.

3.1 Backward Sweeping Strategy

To accelerate dynamic programming convergence, states are updated in reverse order along the spatial horizontal axis x , sweeping backward from $x = W - 2$ down to 0. Algorithm 1 outlines the complete computational process.

Algorithm 1 Dynamic Trajectory Optimization via Value Iteration

```
1: Initialize  $V(x, y, v) \leftarrow 0$  for all  $(x, y, v) \in \mathcal{S}$ 
2: Initialize  $\pi(x, y, v) \leftarrow (0, 0)$ 
3: for iteration = 1 ... 20 do
4:   for  $x = W - 2$  down to 0 do
5:     for  $y = 0 \dots H - 1$  do
6:       if  $\mathbf{T}_{x,y} == 0$  then continue
7:       end if
8:       for  $v = v_{\min} \dots v_{\max}$  do
9:          $Q_{\max} \leftarrow -\infty$ 
10:         $a_{\text{best}} \leftarrow (0, 0)$ 
11:        for each  $a_v \in \{-1, 0, 1\}$  do
12:          for each  $a_y \in \{-1, 0, 1\}$  do
13:             $v' \leftarrow \text{clip}(v + a_v, v_{\min}, v_{\max})$ 
14:             $x' \leftarrow x + v'$ 
15:             $y' \leftarrow y + a_y$ 
16:            if  $x' \geq W \vee y' < 0 \vee y' \geq H$  then continue
17:            end if
18:             $r \leftarrow \text{ComputeReward}(\mathbf{T}_{x',y'}, a_y, v')$ 
19:             $Q \leftarrow r + \gamma V(x', y', v')$ 
20:            if  $Q > Q_{\max}$  then
21:               $Q_{\max} \leftarrow Q$ 
22:               $a_{\text{best}} \leftarrow (a_v, a_y)$ 
23:            end if
24:          end for
25:        end for
26:         $V(x, y, v) \leftarrow Q_{\max}$ 
27:         $\pi(x, y, v) \leftarrow a_{\text{best}}$ 
28:      end for
29:    end for
30:  end for
31: end for
```

4 Implementation Details

The software solution is written in Python utilizing Pygame for display and spatial rendering, combined with NumPy for matrix manipulation.

4.1 Track Generation Geometry

The synthetic track is defined as an S-curve parameterized by a sinusoidal central spine. The baseline midpoint $y_{\text{center}}(x)$ is calculated as:

$$y_{\text{center}}(x) = \left\lfloor \frac{H}{2} + \sin(0.15 \cdot x) \cdot \frac{H}{4} \right\rfloor, \quad x \in [5, W - 5] \quad (11)$$

The solid driving track of uniform thickness $d_{\text{track}} = 10$ cells is produced by setting $\mathbf{T}_{x,y} = 1$ for all spatial offsets $y \in \left[y_{\text{center}} - \frac{d_{\text{track}}}{2}, y_{\text{center}} + \frac{d_{\text{track}}}{2} \right]$.

```

1 def generate_track(W, H):
2     track = np.zeros((W, H), dtype=int)
3     for x in range(5, W - 5):
4         y_center = int(H / 2 + np.sin(0.15 * x) * (H / 4))
5         for y in range(y_center - 5, y_center + 5):
6             if 0 <= y < H:
7                 track[x, y] = 1 # Track area
8     # Set finish line
9     track[W - 6, :] = np.where(track[W - 6, :] == 1, 2, 0)
10    return track

```

Figure 1: Source implementation of sinusoidal track grid generation.

5 Experimental Results and Visual Analysis

Execution of 20 backward value iteration passes yielded full convergence across valid operational track cells.

5.1 Policy Trajectory Profiling

Table 1 summarizes the dynamic system environment configurations and RL hyperparameters applied during testing.

Table 1: System Environment and RL Hyperparameter Configurations

Parameter	Symbol	Value
Grid Screen Dimensions	$W \times H$	66×46 cells
Cell Edge Size	g	15 pixels
Discount Factor	γ	0.95
Velocity Bounds	$[v_{\min}, v_{\max}]$	$[1, 6]$ cells/step
Goal Reward	R_{goal}	+100.0
Collision Penalty	R_{wall}	-50.0
High-Speed Turn Penalty	c_{turn}	0.5

5.2 Key Observed Behaviors

1. **Apex Cutting (Straightening Curves):** Instead of strictly tracking the central spline, the agent utilizes full track width, steering toward inner corners to minimize path distance across horizontal intervals.
2. **Deceleration in High-Curvature Regions:** When entering sharp turns, the policy applies negative acceleration actions ($a_v = -1$) to maintain speeds below $v_{\max}$, avoiding the c_{turn} steering penalty.
3. **Straightaway Acceleration:** Upon exiting turns, the agent consistently selects maximum positive acceleration ($a_v = +1$) to reach $v_{\max} = 6$.

5.3 Final Result Image

The final result is shown below. This image shows how well the raceline is optimized (braking before corners, going near the boundary during apex, etc).

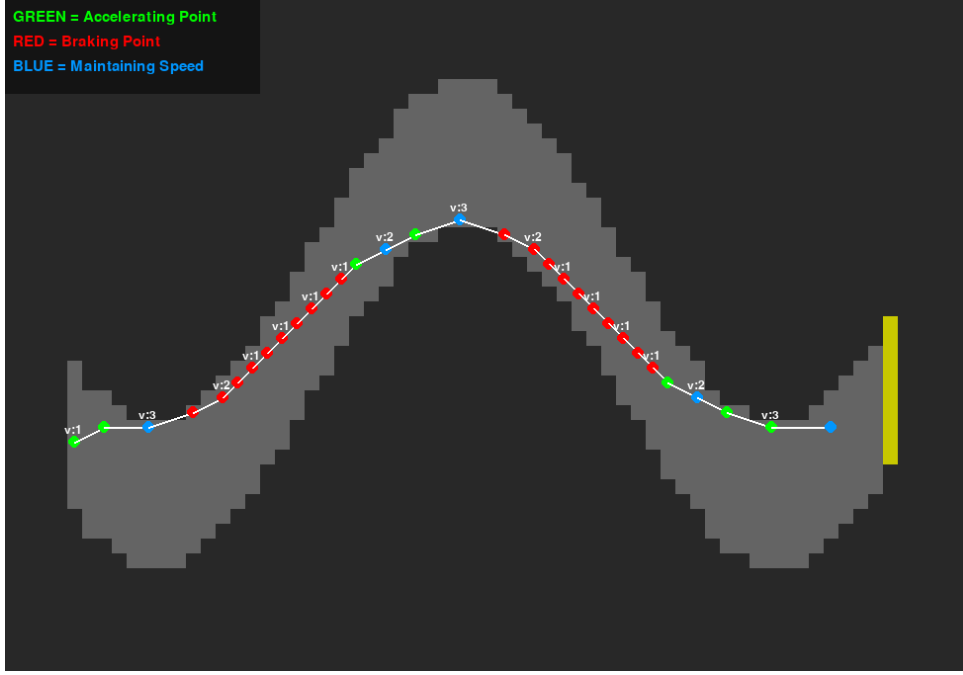


Figure 2: Visual rendering of optimal trajectory line generated via Bellman Value Iteration.

6 Discussion and Limitations

6.1 Absence of Physical Dynamics

The primary limitation of this simplified model is its non-physics-informed nature. Standard vehicle dynamics models incorporate force balances including lateral cornering stiffness, tire friction ellipses ($F_x^2 + F_y^2 \leq (\mu F_z)^2$), weight transfer, and inertia. Here, vehicle handling is modeled purely via dynamic cost penalties (c_{turn}) rather than physical differential equations.

6.2 Discretization Artifacts

Because horizontal movement is constrained to integer velocity jumps ($x_{t+1} = x_t + v_{t+1}$), the agent occasionally skips spatial cells. Higher speed values increase spatial step sizes, which can cause the vehicle to step over narrow apex boundaries or miss fine-grained turning markers.

7 Conclusion and Future Work

This project demonstrates that model-based dynamic programming using Bellman Value Iteration can efficiently calculate optimal paths and speed profiles on continuous track geometries using discrete approximations. Even without explicit physics equations, introducing velocity-dependent turning penalties produces recognizable racing behaviors such as apex cutting, corner deceleration, and straightaway acceleration.

Future work will focus on:

1. Integrating continuous spatial state spaces using Deep Q-Networks (DQN) or Proximal Policy Optimization (PPO).
2. Implementing point-mass or bicycle vehicle dynamics models to enforce realistic acceleration and lateral slip limits.

3. Extending the environment to continuous 2D steering vector controls instead of fixed axis movement.